IMPLEMENTATION CHECKPOINT

Phishing URL Detection Module

Assignment Evidence: Screenshots of Model Evaluation and Experiment Code

Student Name: JEEVITHA L SHETTY
SRN : R23EQ045

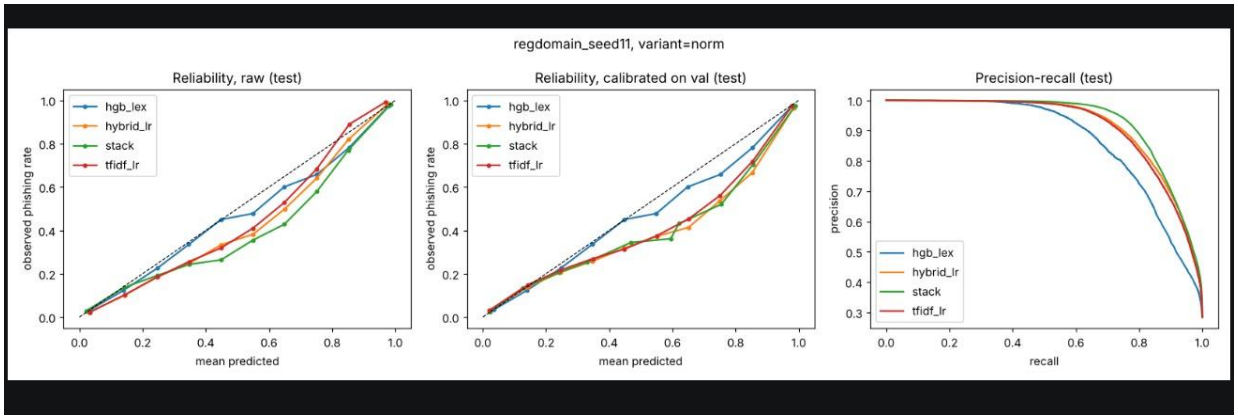
1. Objective of this checkpoint

This document organizes the screenshots supplied for the implementation checkpoint of the Phishing URL Detection module. The evidence covers model reliability/calibration and precision–recall evaluation, shift-scenario analysis, a learning-curve experiment, and the corresponding experiment-runner source code.

The screenshots are presented as supplied. No unseen results, performance values, or successful execution steps are claimed. The current evidence is strongest for experimental analysis and code structure; a separate terminal or application screenshot showing an actual URL input and returned prediction would strengthen proof of end-to-end runtime execution.

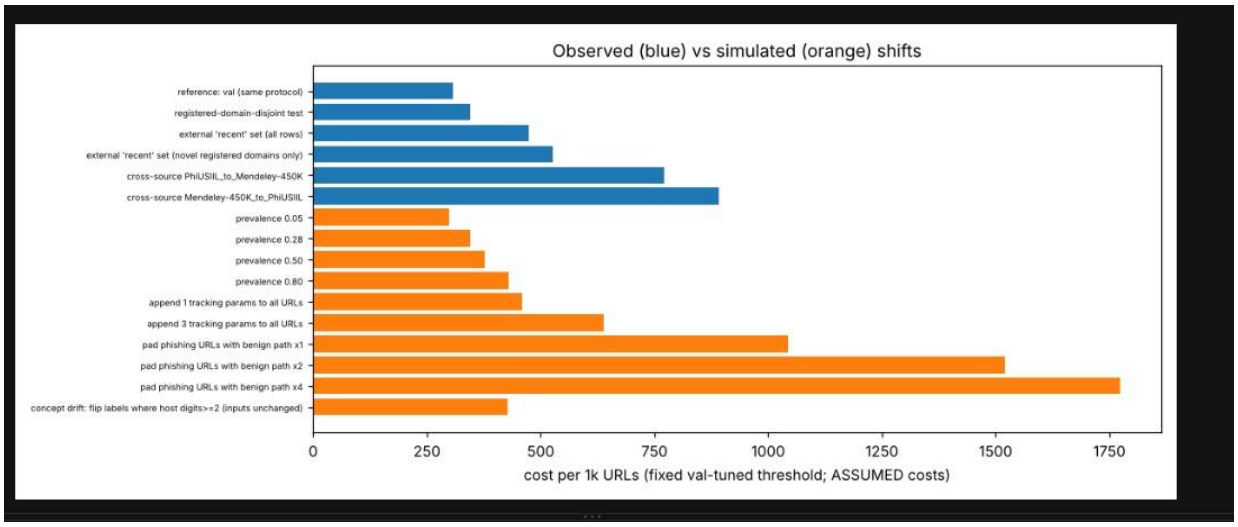
2. Evidence screenshots

Screenshot 1. Model reliability and precision–recall evaluation



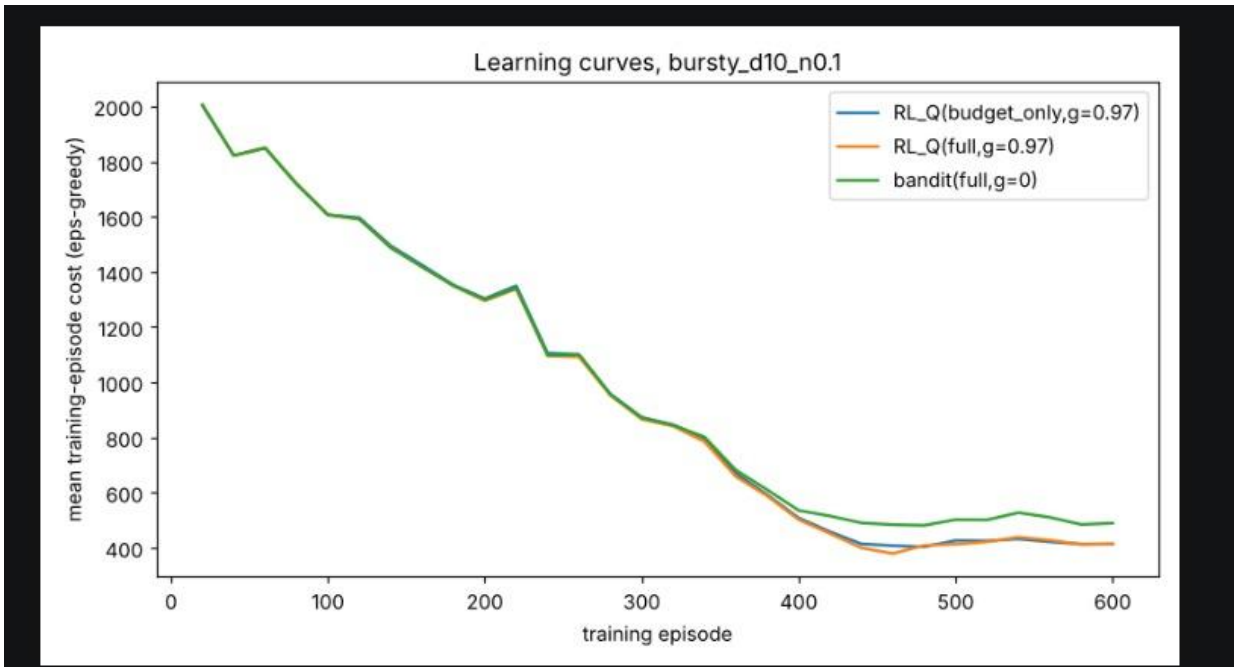
The figure compares hgb_lex, hybrid_lr, stack and tfidf_lr using reliability plots on the test set, reliability after validation calibration, and a precision–recall curve. It documents model evaluation and calibration analysis for the phishing URL detection research. The screenshot itself does not provide exact metric values in text, so no numeric result is inferred here.

Screenshot 2. Observed and simulated distribution-shift cost comparison



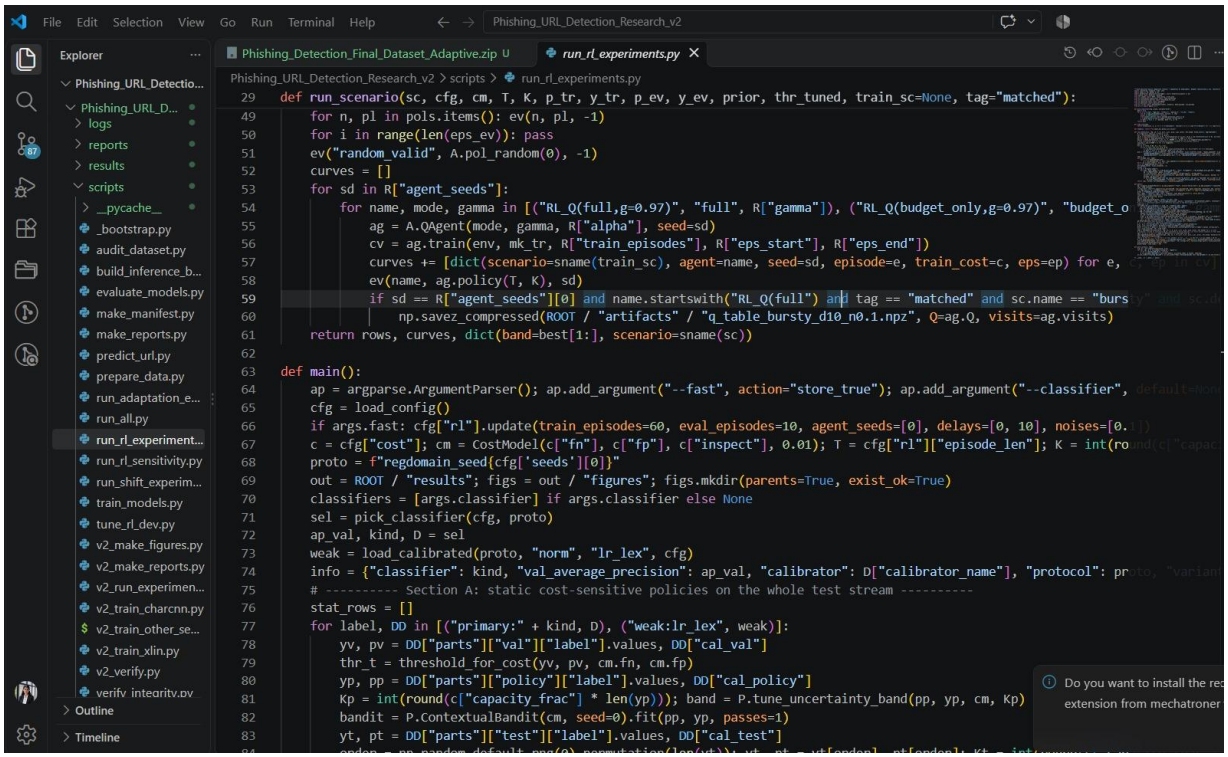
The horizontal bar chart compares reference/held-out datasets and simulated changes, including prevalence changes, tracking-parameter additions, benign-path padding and concept drift. The x-axis describes cost per 1,000 URLs at a validation-tuned threshold and explicitly marks the costs as assumed. These are scenario-analysis results, not measured production costs.

Screenshot 3. Reinforcement-learning / contextual-bandit learning curves



The plot compares mean training-episode cost over episodes for RL_Q policies and a bandit baseline in a scenario labelled bursty_d10_n0.1. The curves show training progress for the reported experiment. This is supporting policy-experiment evidence, not by itself proof that reinforcement learning outperforms the baseline.

Screenshot 4. Source-code view of the experiment runner



The VS Code screenshot shows the run_rl_experiments.py script, including policy evaluation, multiple agent seeds, training-curve collection, and output of a compressed Q-table artifact under a scenario condition. It is code evidence for the experimental workflow; it does not independently demonstrate a terminal run or a URL prediction response.

3. Core functionality demonstrated by the available evidence

- Model comparison and calibration analysis: reliability diagrams and precision–recall curves are shown for multiple candidate classifiers.
- Distribution-shift and robustness scenario analysis: the chart includes external/cross-source conditions and simulated URL transformations; the chart labels scenario costs as assumed.
- Experiment training workflow: the learning-curve figure records episode-level cost across several policy/baseline configurations.
- Implementation evidence: the visible Python script includes evaluation loops, agent seeds, learning-curve collection and conditional artifact saving.

4. Evidence limitations and recommended final screenshot

The supplied screenshots do not visibly show a command being executed in a terminal or a completed URL prediction in the application. If the LMS task specifically requires proof of working core-module execution, capture one additional screenshot after running the project that shows (1) the command or run action, (2) a sample URL input, and (3) the returned phishing/legitimate prediction or evaluation summary. Do not edit or fabricate output values.

The shift chart explicitly describes the costs as assumed; these should not be described as measured production costs. The learning curve alone does not establish that the RL or bandit method is better than the baseline. Any claims should remain limited to the experiment and dataset represented.